// MODULE · AWS FOR DEVELOPERS

Deploy Docker Images on AWS ECS

Everything you need to understand AWS infra — from what it is to deploying your container live. Short, simple, no fluff.

Docker Image → ECR Registry → ECS Task Def → ECS Service → ALB Load Balancer → Public Internet

01 What is AWS?

FOUNDATION

AWS (Amazon Web Services) is a cloud platform by Amazon. Instead of buying servers, you **rent** them from AWS — pay only for what you use.

> **Think of it like this:** Zomato rents a commercial kitchen instead of building one. You rent servers from AWS instead of buying one.

WHY USE AWS?

- No upfront hardware cost
- Scale up/down anytime
- Servers run 24/7 without you managing them physically
- Global — deploy in any country with one click

02 ECR — Elastic Container Registry

STORAGE

ECR is like **Docker Hub**, but private and inside AWS. You push your Docker image here, and ECS pulls it from here to run your container.

> **Analogy:** ECR is a private locker inside AWS where you store your Docker images. Only your AWS account can access it.

BASIC FLOW

```
# Step 1 - Login to ECR
aws ecr get-login-password | docker login --username AWS --password-stdin
YOUR_ECR_URL
# Step 2 - Tag your image
docker tag myapp:latest YOUR_ECR_URL/myapp:latest
# Step 3 - Push to ECR
docker push YOUR_ECR_URL/myapp:latest
```

> **Key point:** ECR is private. ECS already has permission to pull from it — no Docker Hub needed.

03 ECS — Elastic Container Service

CORE

ECS is AWS's service to run **Docker containers**. You tell it what image to run, how much CPU/RAM to give it — ECS handles the rest.

> **Analogy:** ECS is like a container manager on a factory. You give it a blueprint (Task Definition) and it runs containers for you automatically.

3 KEY ECS CONCEPTS

- **Cluster** — The group/environment where your containers run. Like a "project folder" in ECS.
- **Task Definition** — The blueprint. Which image? How much CPU/RAM? Which port? Environment variables?
- **Service** — Keeps your tasks running. If a container dies, it auto-restarts it. Also integrates with Load Balancer.

ECS LAUNCH TYPES

EC2 Launch Type
You manage the servers (EC2 instances). More control, more work. Good for GPU or custom setups.

Fargate (Recommended)
Serverless. AWS manages servers for you. You only define CPU/RAM. **Best for beginners.**

> For most web apps, **use Fargate**. You don't have to manage any EC2 instances.

04 VPC — Virtual Private Cloud

NETWORKING

VPC is your **private network inside AWS**. All your AWS resources (ECS, RDS, ECs) live inside a VPC. It isolates your infra from others.

> **Analogy:** VPC is like a gated society. Your servers are flats inside. Security Group is the guard at the gate who decides who can enter.

SUBNETS INSIDE VPC

Public Subnet
Has internet access. Load balancer lives here. Users can reach it.

Private Subnet
No direct internet access. Your ECS containers and DB live here. Safer.

> AWS gives you a **default VPC** in every region. For most projects, you can use it without creating a new one.

05 Security Group

FIREWALL

Security Group is a **virtual firewall** that controls which traffic is allowed IN (inbound) and OUT (outbound) of your resource.

> **Analogy:** It's the bouncer at a club. You write rules: "Allow HTTP on port 80", "Allow only from this IP". Everything else is blocked by default.

COMMON RULES YOU'LL SET

- **ALB Security Group** → Allow inbound HTTP (80) and HTTPS (443) from anywhere (0.0.0.0/0)
- **ECS Task Security Group** → Allow inbound only from ALB's Security Group on your app port (e.g., 3000)
- **RDS Security Group** → Allow inbound only from ECS Security Group on port 5432 / 3306

> **Key rule:** Never expose your ECS task or database directly to the internet. Let traffic come through ALB only.

06 Target Group

ROUTING

Target Group is the **list of containers/servers** that the Load Balancer sends traffic to. It also does health checks to make sure the container is alive.

> **Analogy:** ALB is the receptionist. Target Group is the list of employees (containers) the receptionist can forward calls to. Health check = "Is this employee available?"

WHAT TO CONFIGURE

- **Target type:** IP (for Fargate tasks)
- **Protocol:** HTTP
- **Port:** Your app port (e.g., 3000)
- **Health check path:** `/` or `/health` — ALB pings this to check if container is alive

> ECS Service automatically registers/deregisters tasks in the Target Group when containers start or stop.

07 ALB — Application Load Balancer

TRAFFIC

ALB sits in front of your containers and **distributes incoming traffic** across all running ECS tasks. It's also the single public-facing entry point.

> **Analogy:** You have 3 Zomato delivery guys (containers). ALB is the dispatcher who fairly assigns orders (requests) to whichever guy is free.

WHAT ALB CAN DO

- Distribute traffic across multiple containers
- Route `/api/*` to backend, `/` to frontend (path-based routing)
- Handle SSL/HTTPS termination (attach an SSL cert)
- Stop sending traffic to unhealthy containers (via Target Group health checks)

> ALB gives you a public DNS URL. Map your custom domain to this URL using Route 53 or any DNS provider.

08 IAM — Identity and Access Management

PERMISSIONS

IAM controls **who can do what** in your AWS account. Every service (ECS, ECs, etc.) needs IAM roles to talk to each other.

> **Analogy:** IAM is the HR department. It gives each employee (service) an ID card with specific permissions. ECS needs an IAM role to pull images from ECR.

KEY ROLES YOU'LL CREATE

- **ecsTaskExecutionRole** → Allows ECS to pull images from ECR and send logs to CloudWatch
- **ecsTaskRole** (optional) → Gives your running container permission to call other AWS services (like S3)

> If your container can't start, 90% of the time it's a wrong IAM role or missing permission. Always attach `AmazonECSTaskExecutionRolePolicy` to your task execution role.

09 CloudWatch — Logs & Monitoring

OBSERVABILITY

CloudWatch collects **logs and metrics** from your AWS services. When your ECS task crashes, you check CloudWatch to see why.

> **Analogy:** It's the CCTV + activity log of your AWS setup. Every `console.log` in your container shows up here.

- In Task Definition, set log driver to: **`awslogs`** to enable container logging
- Go to CloudWatch → Log Groups → `/ecs/your-app-name` to see logs
- Set alarms on CPU/memory usage to get notified before things break

10 Full Deployment — Step by Step

PRACTICE

Here's the exact order of steps when deploying a Docker app to ECS for the first time.

1. **Create ECR Repository**
   Go to AWS ECR → Create repository → Push your Docker image as the push commands provided.
2. **Create ECS Cluster**
   Go to ECS → Clusters → Create Cluster. Choose Fargate. This is your working environment.
3. **Create Task Definition**
   Set your ECR image URL, CPU/RAM (e.g., 0.5 vCPU, 1GB), app port, environment variables, and IAM execution role.
4. **Create Target Group**
   ECS → Target Groups → Create. Type: IP. Port: your app port. Health check: /health or /
5. **Create Application Load Balancer**
   EC2 → Load Balancers → Create ALB. Attach it to public subnets. Create listener on port 80, forward to your Target Group.
6. **Create ECS Service**
   In your Cluster → Create Service. Choose your Task Definition, set desired task count, attach the ALB + Target Group, set Security Groups.
7. **Test via ALB DNS**
   Copy the ALB DNS name → paste in browser. Your app should be live. Point your domain to ALB DNS using CNAME.

AWS — Cloud platform. Rent servers, pay as you go.

ECR — Private Docker Hub inside AWS.

ECS — Runs your Docker containers in the cloud.

VPC — Your private network in AWS.

Security Group — Firewall rules for each resource.

Target Group — List of containers ALB sends traffic to.

ALB — Distributes traffic to your containers.

IAM — Permissions for AWS services.

CloudWatch — Logs and monitoring for your app.